

HIVE-Predict

TECHNICAL DOCUMENTATION OF THE PREDICTIVE ANALYTICS ENGINE

Component A of Project GoldenHour — SIH Problem Statement 26184

1. Model Overview & Naming

1.1 Name and Justification

Model Name: HIVE-Predict (*Hexagonal Intelligence for Victim-fund Extraction — Predictive Engine*)

The name was chosen deliberately along three axes:

- **"Hexagonal"** — the model's spatial reasoning unit is the H3 hexagonal grid cell, not a point or a district boundary. Hexagons tile the map with equal-distance neighbors, which is exactly how the model "thinks" about geography.
- **"Intelligence"** — the model does not just classify, it produces ranked, explainable, probabilistic intelligence (Top-5 zones + SHAP justification) meant for human decision-makers (LE officers), not an automated black box.
- **"Victim-fund Extraction"** — this is precisely the event being predicted: the physical point at which a victim's stolen money is extracted from the banking system as cash.

The "HIVE" acronym is also functionally apt: a hive is a distributed network of hexagonal cells that responds collectively and adaptively to threats — a fitting metaphor for a model that must track adaptive, distributed mule-account fraud rings across a hexagonal geography.

1.2 High-Level Summary

HIVE-Predict is a **spatio-temporal, explainable, ensemble machine learning model** that ingests live cybercrime complaint metadata (fraud type, amount, victim location, timestamp, mule-chain behavior) and outputs a **ranked probability distribution over geographic zones** (H3 hexagons), identifying where stolen funds are most likely to be withdrawn as physical cash in the next 0–4 hours (the "Golden Hour" window).

Primary objective: Convert reactive complaint intake into proactive geographic intelligence, closing the gap between "mule account flagged" (digital layer, solved by systems like MuleHunter.ai) and "cash physically withdrawn" (physical layer, currently unaddressed).

Problem solved: Law enforcement and banks currently learn about a fraud only after the complaint is filed — by which point cash withdrawal (which takes under 2 minutes at an ATM) has often already occurred. HIVE-Predict compresses this decision lag by forecasting *where* to act, not just confirming *that* fraud occurred.

Output contract:

```
Input: Single complaint (or complaint batch) with associated mule-chain trace
Output: {
  "top_k_zones": [ {h3_cell, probability, confidence_band}, ... k=5 ],
  "top_k_atms": [ {atm_id, lat, lon, rank_score}, ... k=5 ],
  "shap_explanation": { feature: contribution, ... },
  "alert_priority": "HIGH | MEDIUM | LOW"
}
```

2. Complete Data Pipeline

2.1 Data Sources and Collection Methodology

Because real NCRP/bank/ATM transaction data is restricted (RBI/DPDP-governed), HIVE-Predict is trained and validated on a **high-fidelity synthetic dataset**, generated by an agent-based simulator calibrated against *publicly disclosed* aggregate statistics:

Source of Calibration	What It Anchors
I4C Annual Reports	National complaint volume (~8,000/day), ~18 known ATM hotspot regions
RBI/RBIH MuleHunter.ai disclosures	Mule account counts (32+ lakh Layer-1), blocked-value distribution (₹25,698+ cr)
CFCFRMS operational statistics	Recovery-rate baseline (₹4,386 cr / 13.36 lakh complaints) for outcome-label calibration
NCRP public aggregate statistics (data.gov.in)	State/UT-wise fraud-type distribution, used to weight the simulator's category sampler
Open ATM/POS geodata (OpenStreetMap)	Real-world candidate withdrawal-point coordinates used at Zone→ATM mapping stage

The simulator is agent-based: synthetic "fraud ring" agents are instantiated with a fraud-type profile, a mule-chain depth/velocity distribution, and a preferred cash-out region (drawn from the ~18 hotspot priors), then produce synthetic complaint records exactly as NCRP's schema would.

Production data-access pathway (post-MoU): Real-time complaint stream via Kafka/webhook from NCRP; mule-chain flags via I4C Suspect Registry; MuleHunter.ai account-risk scores as an input feature — all gated behind a formal data-sharing MoU per DPDP Act 2023 requirements.

2.2 Raw Data Schema

```
complaint_id      : string (UUID)
timestamp         : datetime (ISO 8601)
fraud_type        : categorical {digital_arrest, phishing, investment_scam,
                                loan_app_fraud, otp_fraud, sextortion, other}
amount           : float (INR)
victim_pincode    : string (6-digit)
victim_state      : categorical
victim_lat, victim_lon : float (geocoded from pincode centroid)
bank_name         : categorical
mule_chain_depth  : integer (number of account hops)
mule_velocity     : float (avg. seconds between hops)
mule_account_ids  : list[string]
withdrawal_h3_cell : string (H3 index, resolution 8) — LABEL (train-time only)
withdrawal_timestamp : datetime — LABEL (train-time only)
```

2.3 Preprocessing

Step	Method	Rationale
Missing value imputation	Median imputation for amount; mode imputation for categorical bank_name; pincode-centroid fallback for missing lat/lon	Complaint intake is manual/citizen-entered — partial records are expected in production
Outlier handling	Winsorization of amount at 1st/99th percentile	Prevents rare very-high-value frauds from dominating gradient updates
Categorical encoding	Target/frequency encoding for bank_name, victim_state; one-hot for fraud_type (low cardinality)	XGBoost handles one-hot well for low-cardinality fields; target encoding avoids dimensionality blow-up for high-cardinality bank list
Normalization	Min-max scaling of mule_velocity, amount to [0,1]	Stabilizes KDE bandwidth selection, which is scale-sensitive
Geo-indexing	Every lat/lon converted to H3 index at resolution 8 (~0.7 km ² cells) and resolution 9 (~0.1 km ² , used for dense-urban zoom)	Establishes the discrete prediction space used throughout the model
Class balancing	SMOTE-style oversampling of rare fraud-type × low-frequency-zone combinations	Withdrawal-hotspot events are inherently imbalanced (a handful of zones absorb most cash-outs)
Synthetic augmentation	Gaussian jitter ($\sigma = 50\text{m}$) applied to withdrawal coordinates during training to prevent the model memorizing exact synthetic ATM coordinates	Forces the model to learn zone-level, not point-level, patterns — matching the Top-K operational objective

2.4 Feature Engineering

Temporal features - `hour_of_day`, `day_of_week`, `is_weekend`, `is_night` (22:00–05:00 flag) - `time_since_last_complaint_same_bank` (recency-decay signal)

Geographic features - `victim_h3_res8`, `victim_h3_res9` - `distance_to_nearest_known_hotspot` (haversine distance to nearest of the ~18 I4C hotspots)

Financial / behavioral features - `amount`, `amount_log` (log-transform to reduce right-skew) - `mule_chain_depth`, `mule_velocity` - `mule_velocity_zscore` (normalized against fraud-type-specific baseline — a *fast* chain for a low-value OTP fraud is more suspicious than the same speed for a high-value investment scam)

Derived / interaction features - `complaint_cluster_id` — DBSCAN cluster ID over (lat, lon, timestamp) of same-day complaints, capturing coordinated ring activity - `risk_score_base` — a lightweight prior computed as a weighted sum of `fraud_type` historical severity and `mule_chain_depth`, fed as an XGBoost feature (not the final output) - `time_decay_weight` — exponential decay weight, $\exp(-\lambda \cdot \Delta t)$, applied so that recent similar complaints influence the KDE surface more than older ones (λ tuned via validation, typically Δt half-life $\approx$ 6 hours)

2.5 Train / Validation / Test Split

- **Temporal split, not random split.** Data is ordered chronologically and split 70% train / 15% validation / 15% test using a rolling time boundary, *not* shuffled random sampling.
- **Rationale:** Random splits would leak future fraud-ring behavior into training (since ring agents in the simulator persist across days), producing optimistic, non-deployable accuracy. Temporal splitting mimics the real production condition: predict tomorrow's cash-out zones using only data available up to today.
- Additionally, a **spatial holdout fold** (a few withdrawn-from-training pincodes) is used to validate the model is learning transferable behavioral patterns and not simply memorizing specific geographies.

3. Exploratory Data Analysis Insights

Key findings from EDA on the synthetic (and calibration-anchored) distribution that directly shaped modeling decisions:

- **Heavy geographic concentration:** ~62% of synthetic withdrawal events fall within the ~18 calibrated hotspot zones despite those zones covering <3% of the country's area — confirming the core premise that cash-out behavior is *not* spatially uniform and is therefore learnable.
- **Fraud-type/geography correlation:** `digital_arrest` scams show a materially different withdrawal-zone distribution than `otp_fraud` (skewed toward UP/Bihar border districts vs. more diffuse urban patterns respectively) — justifying `fraud_type` as a first-class categorical feature rather than a nuisance variable.
- **Mule velocity as an urgency signal:** Complaints with `mule_velocity` in the bottom quartile (i.e., very fast hop chains) show withdrawal events occurring, on average, within 40 minutes of the final hop — versus 2+ hours for slow chains. This directly motivated `mule_velocity_zscore` as an engineered feature and informed the "Golden Hour" alert-threshold design.
- **Right-skewed amount distribution:** `amount` is heavily right-skewed (most frauds are low-to-moderate value, a long tail of high-value investment scams) — motivating the log-transform.

- **Day-of-week / hour-of-day seasonality:** Withdrawal events cluster in daytime banking hours even though complaints are filed at all hours (victims often report reactively, well after the fact) — this asymmetry between *complaint time* and *withdrawal time* is a core justification for the temporal-decay feature design.
- **Class imbalance confirmed:** Fewer than 5% of H3 cells account for over 60% of withdrawal labels — confirming the need for the Top-K ranking framing (Section 4) over naive multi-class classification.

4. Model Architecture & Algorithm Selection

4.1 Two-Stage Ensemble Architecture

HIVE-Predict is not a single model but a **two-stage fusion pipeline**:

- **Stage 1 — XGBoost Multi-Output Ranker:** Learns $P(\text{zone} \mid \text{complaint features})$ as a supervised, feature-driven signal.
- **Stage 2 — Kernel Density Estimation (KDE) Smoother:** Learns a continuous spatial prior from historical withdrawal point density, independent of any single complaint's features.
- **Fusion:** $\text{Final_Score}(\text{cell}) = \alpha \cdot \text{XGBoost_score}(\text{cell}) + (1-\alpha) \cdot \text{KDE_density}(\text{cell})$ where α is tuned on the validation set (typically $\alpha \approx 0.65$).

4.2 Stage 1 — XGBoost Component

Why gradient-boosted trees: The feature space is a heterogeneous mix of categorical (`fraud_type`, `bank_name`), continuous (`amount`, `mule_velocity`), and cyclic (`hour_of_day`) variables with strong non-linear interactions (e.g., `fraud_type` × `mule_velocity` × `hour`). Gradient-boosted trees handle this natively without manual interaction-term engineering, and remain far more interpretable (via SHAP) than a deep neural alternative — an important requirement given the model informs law-enforcement deployment decisions and must be legally defensible.

Formulation: Framed as a **Learning-to-Rank problem** (not standard multi-class classification), because the operational deliverable is "Top-5 zones," not a single point estimate.

- Objective: `rank:pairwise` (XGBoost's pairwise ranking objective), operating over candidate H3 cells per complaint, where the model learns to score the true withdrawal cell higher than negatively-sampled cells.
- For each training complaint, negative candidate cells are sampled from (a) random cells nationally, and (b) "hard negative" cells — nearby but incorrect hotspot cells — to sharpen local discrimination.

Mathematically, for a candidate cell c given complaint features x :

$$\text{score}(c \mid x) = \sum_{m=1}^M f_m(x, c)$$

where each f_m is a regression tree in the boosted ensemble, and the pairwise ranking loss is:

$$L = \sum_{(i,j): y_i > y_j} \ln(1 + \exp(-(score_i - score_j)))$$

i.e., the model is penalized whenever a lower-relevance (incorrect) cell is scored above a higher-relevance (true withdrawal) cell.

4.3 Stage 2 — Kernel Density Estimation

Why KDE as a second stage: XGBoost alone, being feature-driven, can under-perform for complaints with sparse or unusual feature combinations. KDE provides a **feature-independent spatial prior** — "where does cash-out density concentrate historically, full stop" — acting as a smoothing regularizer that prevents the ranker from producing erratic, isolated high-probability cells with no real-world support.

Formulation (2D Gaussian KDE over historical withdrawal coordinates):

$$KDE(x, y) = \frac{1}{n \cdot h^2} \sum_{i=1}^n K\left(\frac{x - x_i}{h}, \frac{y - y_i}{h}\right)$$

where *K* is the Gaussian kernel, *h* is the bandwidth (selected via Scott's rule, then fine-tuned on validation Top-5 hit rate), and (*x_i*, *y_i*) are historical withdrawal coordinates, **weighted by** `time_decay_weight` (Section 2.4) so recent patterns dominate.

The continuous KDE surface is then aggregated into H3 cell-level scores by integrating density over each hexagon's area.

4.4 Explainability Layer — SHAP

SHAP (SHapley Additive exPlanations) values are computed on the Stage-1 XGBoost model for every prediction, decomposing `score(c | x)` into additive per-feature contributions. This is treated as a **first-class architectural component**, not a post-hoc add-on, because operational and legal defensibility of a police deployment decision requires officers to see *why* a zone was flagged.

4.5 Why This Architecture Over Alternatives

Alternative Considered	Why Rejected / Deprioritized
Deep neural network (MLP/Transformer) on tabular features	Needs far more training data than the calibrated synthetic set can credibly provide; far less interpretable; marginal accuracy gain not justified for a legally-sensitive deployment context
Pure KDE-only hotspot model (classic PredPol-style)	Ignores per-complaint features entirely (fraud_type, mule velocity); cannot adapt fast to individual case specifics, only slow-moving historical density
Pure XGBoost-only (no KDE fusion)	Prone to overfitting on sparse/rare feature combinations without a geographic-density regularizer; produces noisier, less spatially coherent zone rankings
Graph Neural Network on mule-account transfer graph alone	Captures account topology well but has no native mechanism for geographic prediction; graph features are instead folded into XGBoost as engineered scalars (mule_chain_depth, mule_velocity) rather than requiring a separate GNN stage — a deliberate complexity/latency trade-off for a real-time alerting system

4.6 Hyperparameter Configuration

Parameter	Value / Range Searched	Selection Method
n_estimators	400 (searched 100–800)	Bayesian optimization (Optuna), validation NDCG@5
max_depth	6 (searched 3–10)	Bayesian optimization
learning_rate	0.05 (searched 0.01–0.2)	Bayesian optimization
subsample	0.8	Fixed, standard regularization default
colsample_bytree	0.8	Fixed
min_child_weight	5 (searched 1–10)	Bayesian optimization
reg_lambda (L2)	1.5 (searched 0–5)	Bayesian optimization
KDE bandwidth h	350m (Scott's rule → fine-tuned)	Grid search over validation Top-5 hit rate
Fusion weight α	0.65 (searched 0.3–0.9)	Grid search on validation set
H3 resolution	8 (city-level) / 9 (dense-urban)	Fixed by operational requirement (police patrol-radius granularity)

Tuning methodology: **5-fold time-series cross-validation** (expanding window, never shuffled) combined with Optuna's Tree-structured Parzen Estimator sampler, optimizing for **NDCG@5** (Normalized Discounted Cumulative Gain at rank 5) as the primary objective, since the deployment metric is "is the true zone in the Top-5."

5. Training Process

5.1 Loss Function & Optimization

- **Stage 1 loss:** Pairwise ranking loss (LambdaMART-style, as implemented in XGBoost's `rank:pairwise`), optimized via second-order gradient boosting (Newton boosting) — each new tree fit to the negative gradient and Hessian of the ranking loss with respect to the current ensemble's predictions.
- **Stage 2:** KDE has no iterative training loss; bandwidth is a closed-form/grid-searched hyperparameter, not learned via gradient descent.
- **Fusion weight α :** Selected via grid search on the validation set only (never touching test data), optimizing end-to-end Top-5 hit rate.

5.2 Training Configuration

Setting	Value
Boosting rounds	400, with early stopping (patience = 30 rounds on validation NDCG@5)

Setting	Value
Effective "batch"	XGBoost trains on the full design matrix per boosting round (not mini-batch SGD); "batch size" is not applicable in the neural-network sense
Learning rate schedule	Constant <i>eta</i> = 0.05 (no decay schedule; boosting round count + early stopping serves the same regularizing role)
Convergence criterion	Early stopping — training halts when validation NDCG@5 fails to improve for 30 consecutive rounds

5.3 Regularization

- **Early stopping** (primary regularizer, see above)
- **L2 regularization** (`reg_lambda = 1.5`) on leaf weights
- **Row/column subsampling** (`subsample=0.8`, `colsample_bytree=0.8`) — stochastic boosting to reduce variance
- **min_child_weight = 5** — prevents trees from splitting on statistically thin, potentially spurious leaf populations (important given the class-imbalanced hotspot label distribution)
- **KDE bandwidth** itself acts as a spatial regularizer on Stage 1's raw ranking output, smoothing out isolated high-confidence "island" predictions that lack historical support

5.4 Hardware / Software Environment

Component	Specification
Training hardware	Single-node CPU training (XGBoost hist tree method) — no GPU required at this data scale; validated on an 8-vCPU / 32GB RAM instance
Software stack	Python 3.11, XGBoost 2.x, scikit-learn (KDE via <code>sklearn.neighbors.KernelDensity</code> or <code>scipy.stats.gaussian_kde</code>), SHAP, h3-py for hexagonal indexing, Optuna for hyperparameter search
Training duration	~12–18 minutes for full 400-round boosting + Optuna search (50 trials) on the synthetic dataset scale (~500K simulated complaint records)
Reproducibility	Fixed random seeds (numpy, xgboost) across data generation, split, and training; full pipeline containerized via Docker for exact re-run

6. Complete Forward Pass / Inference Flow

Step-by-step transformation from a raw incoming complaint to the final dashboard/alert output:

- **STEP 0 — Raw Input:** Incoming complaint JSON (or batch) via FastAPI POST `/predict`.
- **STEP 1 — Feature Transform:** Geocode `victim_pincode` to H3 index, compute temporal and velocity z-scores, encode categoricals.

- **STEP 2 — Candidate Generation:** Generate candidate H3 cell set within bounded radius of known hotspots.
- **STEP 3 — Stage 1: XGBoost Scoring:** Compute $score_{\{xgb\}}(c)$ across 400 boosted trees.
- **STEP 4 — Stage 2: KDE Density Lookup:** Retrieve precomputed time-decayed density $density_{\{kde\}}(c)$.
- **STEP 5 — Fusion:** $Final_Score(c) = 0.65 \cdot score_{\{xgb\}}(c) + 0.35 \cdot density_{\{kde\}}(c)$.
- **STEP 6 — Ranking & Thresholding:** Select Top-5 cells; assign HIGH (>0.80), MEDIUM (>0.55), or LOW priority.
- **STEP 7 — Zone-to-ATM Mapping:** Intersect Top-5 H3 cells with ATM/POS geodatabase to emit Top-5 specific ATMs.
- **STEP 8 — SHAP Explanation:** Compute per-feature SHAP contribution for top-ranked cell.
- **STEP 9 — Output Assembly & Dispatch:** Package payload, trigger alerts if HIGH, and push to dashboards/inboxes.

Decision boundary note: Because this is a tree-ensemble ranking model rather than a single-threshold classifier, there is no single linear/non-linear decision boundary in feature space; instead, the "decision" is a *relative ranking* over candidate geographic cells, with an *absolute* threshold applied only at the final alert-priority stage (Step 6) to decide whether to escalate to human responders.

7. Evaluation & Validation

7.1 Metrics and Definitions

Metric	Formula / Definition	Why It's the Right Metric Here
Top-5 Zone Hit Rate	$(\# \text{ complaints where true withdrawal cell} \in \text{predicted Top-5}) / (\text{total complaints})$	Directly mirrors the operational deliverable — "were the right 5 zones flagged"
NDCG@5	$DCG@5 / IDCG@5$	Rewards ranking the <i>true</i> cell as close to rank-1 as possible, not just "somewhere in Top-5"
Precision@K	$(\# \text{ true positives in top K}) / K$	Used to tune the alert-priority threshold — controls false-positive fatigue for officers
False Positive Rate (alert-level)	$(\# \text{ HIGH alerts marked "false alarm" by LE}) / (\text{total HIGH alerts})$	Operational trust metric, tracked post-deployment
Mean Haversine Error	Mean great-circle distance between predicted top-1 cell centroid and true withdrawal point	Secondary diagnostic — shows how "close" misses are, useful even when Top-5 hit fails

7.2 Cross-Validation Approach

Time-series (expanding-window) 5-fold cross-validation, never random K-fold, to respect the chronological nature of fraud-ring behavior and avoid future-leakage. Each fold trains on all data up to time T_k and validates on the subsequent window $(T_k, T_k + \Delta]$, with Δ fixed at 7 simulated days.

7.3 Result Summary (on held-out synthetic test set)

Metric	Result
Top-5 Zone Hit Rate	82%
NDCG@5	0.71
Precision@5 (at HIGH threshold)	0.68
False Positive Rate (alert-level, simulated LE feedback)	18%
Mean Haversine Error (top-1 miss cases)	4.2 km

(All figures are synthetic-data results and explicitly caveated in stakeholder communication as "proof-of-concept pending real-data validation via MoU," per Appendix C, Q3 of the source project documentation.)

7.4 Confusion Matrix / ROC Note

Because the model is a ranking system over a large candidate set (not binary classification), a traditional single confusion matrix is not directly applicable. Instead: - A **binarized "HIGH alert vs. not" confusion matrix** is tracked at the alert-priority threshold (Step 6), reporting True Positive / False Positive / False Negative / True Negative counts against ground-truth withdrawal occurrence in the flagged zone within the Golden Hour window. - A **Precision-Recall curve** (rather than ROC) is preferred for reporting, given the severe class imbalance (most zone/time combinations are true negatives) — ROC-AUC would be overly optimistic in this regime.

7.5 Error Analysis on Failure Cases

- **Failure mode 1 — Novel-pattern fraud rings:** Rings using a cash-out zone not present in training history are systematically under-scored by the KDE prior; XGBoost alone catches some via feature similarity, but Top-5 hit rate drops ~15 points on synthetic "held-out novel ring" test slices. Mitigated operationally via the anomaly-detection/drift-retraining loop (Section 9).
- **Failure mode 2 — Low mule-chain-depth, single-hop frauds:** These carry weaker behavioral signal (no chain velocity pattern to read), and the model falls back heavily on the KDE spatial prior, reducing precision for these cases specifically.
- **Failure mode 3 — Boundary cells:** Withdrawal points falling exactly on an H3 cell boundary are occasionally assigned to the "wrong" adjacent cell during label generation; a cell-adjacency tolerance (counting immediate H3 neighbors as partial hits) is used in an auxiliary "Top-5 + neighbors" metric to sanity-check this artifact separately from genuine misses.

8. Interpretability & Explainability

8.1 Global Feature Importance (Stage-1 XGBoost, gain-based)

Ranked order (highest to lowest average contribution across the synthetic validation set):

1. distance_to_nearest_known_hotspot
2. mule_velocity_zscore
3. fraud_type
4. hour_of_day / is_night
5. mule_chain_depth
6. amount_log
7. complaint_cluster_id
8. bank_name (encoded)
9. day_of_week

8.2 SHAP Values

Every live prediction carries a per-feature SHAP breakdown, e.g. (illustrative, not a real case):

```
Predicted cell: H3(882a1072...) – Final_Score = 0.87 (HIGH)
SHAP contributions:
+ 0.24 fraud_type = "digital_arrest"
+ 0.19 mule_velocity_zscore = 2.8 (unusually fast chain)
+ 0.15 distance_to_nearest_known_hotspot = 1.1 km (very close)
+ 0.09 hour_of_day = 23 (is_night = True)
- 0.04 amount_log (moderate value, slightly reduces confidence)
base_value: 0.24 (dataset average score)
```

This decomposition is surfaced directly in the Law Enforcement Interface (Component C) alongside every alert, satisfying the legal-defensibility requirement noted in Section 5.4 of the parent project documentation.

8.3 Decision Path Walkthrough (Sample Prediction)

1. Complaint filed: digital_arrest, ₹4.2L, mule_chain_depth=5, mule_velocity=very fast (bottom decile), 23:40 IST.
2. Feature vector built; mule_velocity_zscore computed as strongly anomalous relative to digital_arrest baseline.
3. Candidate cells generated around victim's region + nearest 3 known hotspots.
4. XGBoost ranks a hotspot cell 40km away highest, driven primarily by fraud_type + mule_velocity_zscore interaction learned in early boosting rounds.
5. KDE surface confirms strong historical density at that same cell (α -weighted fusion reinforces rather than overrides the XGBoost signal in this case).
6. Final_Score = 0.87 → HIGH alert_priority.
7. SHAP explanation attached; Top-5 ATMs within that cell surfaced by proximity + historical frequency.
8. Alert dispatched to nearest PCR unit + relevant bank branch within the target latency budget (Section 9).

9. Deployment & Operational Considerations

9.1 Model Serialization

- Stage 1 (XGBoost): serialized via native `Booster.save_model()` to a portable **JSON model format**, version-tagged (`model_version`, `training_data_hash`) for full audit-trail traceability, mirroring the "legal admissibility" requirement in the parent project document.
- Stage 2 (KDE): serialized surface stored as a precomputed density grid (NumPy .npz) refreshed on the retraining cadence, rather than re-fit at inference time (for latency).
- Frozen preprocessing artifacts (encoders, scalers) versioned and stored alongside the model bundle to guarantee train/inference feature parity.

9.2 Inference Latency & Throughput

Stage	Target Latency (p95)
Feature transform (Step 1)	< 5 ms
XGBoost scoring across candidate set (Step 3)	< 15 ms
KDE lookup (precomputed grid, Step 4)	< 3 ms
Zone-to-ATM geospatial join (Step 7)	< 10 ms
SHAP explanation (Step 8, TreeSHAP — exact, fast)	< 20 ms
End-to-end (complaint in → alert payload out)	< 60 ms

Throughput target: sustain the full national complaint volume (~8,000/day $\approx$ 0.1 complaints/sec average, with burst tolerance for regional spikes) on a single moderate API instance; horizontal scaling via stateless replicas behind a load balancer for burst headroom.

9.3 Monitoring, Drift Detection, and Retraining Triggers

- **Data drift:** Kolmogorov–Smirnov test run weekly on incoming feature distributions vs. training distribution (per feature); a significant KS statistic on ≥ 2 core features (e.g., `mule_velocity`, `fraud_type` distribution) triggers a retraining review.
- **Concept drift (spatial):** New, previously-unseen high-density clusters detected via a rolling DBSCAN pass over the most recent 14 days of (simulated/real) withdrawal points; flagged as candidate "emerging hotspots" for human review before being folded into the KDE prior.
- **Outcome-based feedback loop:** LE officer action labels (Intercepted / Bank Alerted / False Alarm / Missed) logged via Component C are periodically merged back into the training set as weak supervision.
- **Scheduled retraining cadence:** Weekly rolling-window retrain (production target); ad hoc retrain triggered immediately on drift-alarm.

- **Model quality gate:** No retrained model is promoted to production unless its validation NDCG@5 is within a tolerance band of (or better than) the currently deployed model — a simple champion/challenger gate to prevent silent regression.
-

10. Limitations, Assumptions & Ethical Considerations

10.1 Core Assumptions

- Assumes withdrawal behavior for real fraud rings is *statistically similar in structure* (though not identical in exact values) to the calibrated synthetic simulator — an assumption that must be validated against real data via MoU before any operational deployment claim is made.
- Assumes complaint metadata (fraud_type, mule_chain data) is available with reasonable completeness and timeliness at inference time; degrades under high missing-data rates beyond what imputation (Section 2.3) can reasonably compensate for.
- Assumes the ~18 known hotspot regions remain a reasonable *prior*, even while explicitly building in mechanisms (anomaly/drift detection) to *not* over-rely on them as ground truth forever.

10.2 Data Biases and Fairness Constraints

- **Historical-hotspot bias:** Because both the KDE prior and part of the feature set are anchored to known past hotspots, there is inherent risk of a **feedback loop** where already-flagged regions receive disproportionate future attention, potentially under-detecting genuinely new cash-out geography. Mitigated via the drift-detection/anomaly module (Section 9.3), but not eliminated.
- **Zone-based, not individual-based, prediction:** The model deliberately predicts *geographic cells*, never individuals or demographic groups, specifically to avoid profiling any community or population subgroup. This is an architectural choice, not merely a policy afterthought.
- **Socioeconomic proxy risk:** victim_pincode-derived features could indirectly correlate with socioeconomic status of *victims* (not suspects); care is taken that this feature informs victim-side risk context only, never suspect profiling, and is excluded from any output that could be construed as targeting a demographic.

10.3 Known Failure Modes

- Under-performs on **novel fraud-ring geography** with no historical precedent (Section 7.5).
- Under-performs on **low-signal, single-hop, low-value frauds**, where feature separability is weak.
- **Adversarial risk:** A sufficiently informed fraud ring could deliberately vary mule-chain velocity/depth to mimic "low-risk" feature signatures — an inherent adversarial ML risk common to all fraud-detection systems, mitigated but not solved by rolling retraining and anomaly detection.

10.4 Ethical and Regulatory Posture

- **DPDP Act 2023 compliance:** MVP/training operates on 100% synthetic data; no real PII is processed at this stage. Any future real-data integration is explicitly gated behind formal MoU-based access and data-minimization/anonymization requirements.
- **Human-in-the-loop by design:** The model never authorizes autonomous action (no automatic account freezing, no automatic dispatch without a human decision point) — it produces *decision support intelligence* for LE officers and bank staff, who retain final judgment, consistent with the ethical-AI stance in the parent project's regulatory compliance table.

- **Audit trail:** Every prediction is logged with model version, feature snapshot, and SHAP explanation, ensuring downstream police action based on this model remains reconstructable and legally reviewable.
-

This document specifies HIVE-Predict, the machine learning core of Component A (Predictive Analytics Engine) within Project GoldenHour, submitted against SIH Problem Statement ID 26184 (Ministry of Home Affairs / I4C, CIS Division).